April 8th, 2026
Sophia Rybnik

Measuring Soccer Player Similarity via Graph Neural Networks

1) Introduction: Project Overview

The goal of this project is to model football (soccer) player similarity. I've decided to first approach this question of player similarity from a passing decision approach, using StatsBomb event data enriched with 360 freeze-frame positional information. Each training example corresponds to a single successful pass event, where the model observes the spatial configuration of all visible players at the moment of the pass and predicts the end location of the ball. More specifically, the task can be expressed as:

$$\textbf{Players} + \textbf{spatial configuration at time } t \rightarrow \textbf{location of ball at time } t + \varepsilon$$

After taking some time to understand the available data, the spatial configurations of the players could naturally be modeled in a graph-based representation, where players are encoded as nodes and their interactions are edges. This representation can be used as input into a Graph Neural Network (GNN), which uses message passing as a way to iteratively update node representations by exchanging information with their neighbours. The aim of the model is to learn to predict where the ball carrier will pass next. More formally,

$$\textbf{Graph of field at pass time} \rightarrow \mathbb{R}^2$$

Ultimately, this work is a step toward quantifying how players play, enabling comparisons in style, strategy, and decision-making across matches and competitions. While the model as it is currently set up aims to predict likely pass destinations given a game state, the idea is to extend the downstream applications and provide a foundation for assessing similarity between players and tactical patterns.

2) Data & Data Processing

The data pipeline is responsible for transforming the raw StatsBomb JSON files into structured graph representations to be used as input for training a GNN. The pipeline design (Figure 1) is modular, reproducible, and decoupled from model training such that the graph construction is performed only once and can be reused across model iterations.

2.1 Raw Data Ingestion

We start off by loading the raw StatsBomb JSON files (currently downloaded directly from the open source data, not through the API), including:

- Event-level data (i.e., a pass, carry, shot, etc)
- 360 freeze-frame data containing the field positions of all visible players at the event time

The events and freeze frames are joined using StatsBomb's unique event identifiers. Only events with 360 data are retained since we consider only training examples with full spatial context.

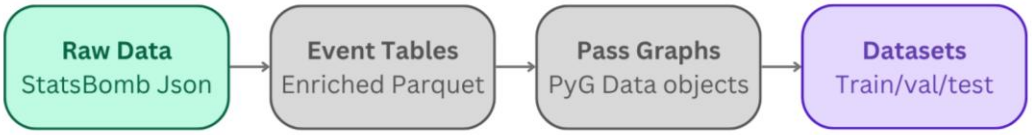

Figure 1 — Data processing pipeline

2.2 Canonical Event Tables

This step isolates data extraction and cleaning from graph construction, which ensures efficiency and reproducibility. The raw JSON files are transformed into canonical tabular formats as follows:

1) Iterate through all available matches
2) Flatten nested JSON fields (player, team, event type, locations)
3) Enrich each event with its associated 360 freeze-frame
4) Save resulting data frames down to parquet files
    - A full “enriched” event table
    - A pass-only subset used downstream

2.3 Freeze-frame to Graph Conversion

This step of the data processing is where the tabular event data gets transformed into a graph representation, as outlined in Figure 2. Each entry in the pass-only parquet file corresponds to a single passing action, identified by meta data including event ID, competition, season, etc. The freeze frame column stores information about the players visible in the frame at the moment of the pass — each record contains the player's pitch location, whether they are the actor (ball carrier), a teammate, or a goalkeeper.

In summary, at the moment of the pass, the following information is known:

- The actor (event-making player)
- The event type
- The positions and roles of all visible players in the freeze frame
- The end location of the ball

This formulation explicitly models passing as a conditional spatial decision problem: given the spatial configuration at pass time, where does the ball end up? In this, we assume that the spatial configuration at the moment of the pass contains sufficient information to predict the pass outcome, without explicitly modeling prior actions or future movement. It is important to note that only the identifier of the actor player is known, all other players are anonymized. For this reason, we will create event-level graphs rather than player-level graphs.

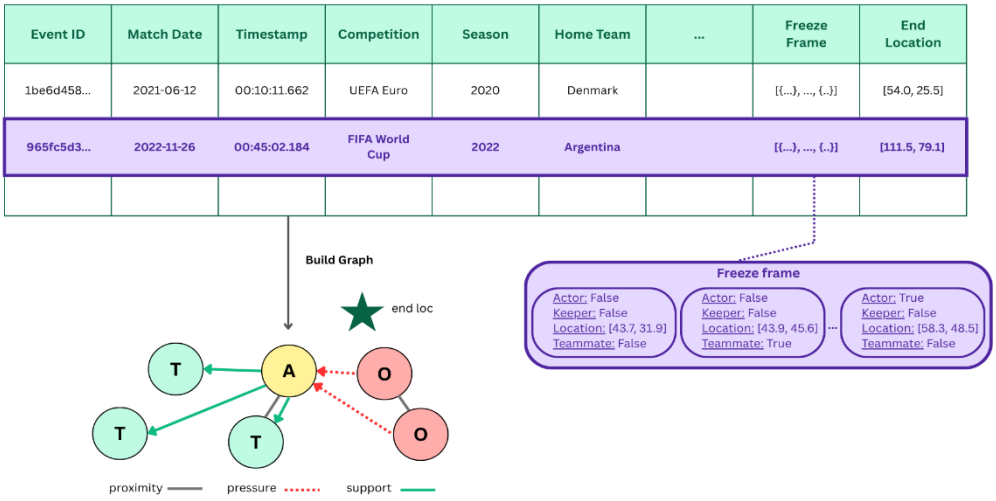

Figure 2 — From tabular event data to graph representation

Each event is transformed into a graph, where the nodes represent individual players and edges encode the spatial relationships between them. The end location from the event table provides the pass destination label used during training. Three types of edges are constructed:

1. Proximity edges: Undirected edges between players within a distance threshold
2. Pressure edges: Directed edges from defenders to the actor
3. Support edges: Directed edges from the actor to forward-positioned teammates

Refer to the appendix for further details about node and edge construction. The resulting graph structure can be visualized as follows:

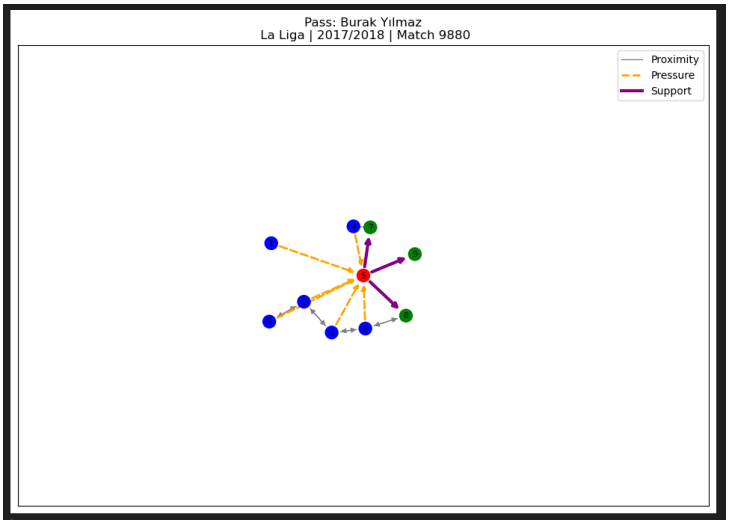

Figure 3 — Graph representation of player state

In total there are 281860 recorded pass events. Following a 70/15/15 split, this results in 196058 train graphs, 42349 validation graphs and 43224 graphs held out for testing.

2.4 Normalization

To ensure stability and keep inputs across matches within comparable ranges, player positions and edge distances are all normalized to (0, 1). For player positions, we use StatsBomb field size normalize the coordinates as $\left(\frac{x}{120}, \frac{y}{80}\right)$. Edge distances are normalized by the pitch diagonal, which represents the maximum possible pass distance.

2.5 Graph Serialization

Once constructed, graphs are stored in data/graphs/ to avoid repeated graph construction. This design choice enables fast experimentation and hyperparameter tuning and ensures deterministic dataset splits. Each serialized graph contains all information required for training, evaluation, and visualization.

3) Methodology

3.1 Model Architecture

The pass destination prediction task is formulated as spatial classification over a discretized pitch grid. Each passing situation is represented as a player graph $G = (V, E)$, where nodes V correspond to players on the pitch and edges E encode spatial relationships between them. The *PassPredictionGNN* model takes this graph as input and outputs a probability distribution over N=grid_x ×grid_y pitch cells.

The model architecture consists of four components, as visualized in Figure 4. First, a linear projection layer maps raw node features into a shared dimensional space. Second, an event type embedding conditions the model on the action being performed. This embedding is added directly to the actor node's representation, informing the model of which player has the ball and what type of event is occurring before any message passing takes place. Third, two stacked GINEConv layers propagate information across the graph. Unlike standard graph convolutions, GINEConv conditions each message on the edge attributes connecting two nodes, allowing the model to reason about the nature of each relationship, i.e., not just proximity, but whether a neighbour is applying pressure or offering support. Refer to the appendix for more details about the multi-dimensional edge-aware message passing mechanics. After two rounds of message passing, the actor node's embedding has absorbed contextual information from its full two-hop neighbourhood. Finally, only the actor's updated embedding is passed to the prediction head. The model is trained with a custom loss function to compute the probability of a pass landing in each cell (refer to appendix for more details), and outputs a distribution over the discretized pitch space.

3.2 Hyperparameter Tuning

Hyperparameters were optimised using Bayesian optimisation via the Optuna framework. The Tree-structured Parzen Estimator (TPE) sampler was used, which builds a probabilistic model of the

search space from completed trials and focuses subsequent sampling toward regions of low validation loss. This is considerably more efficient than grid search. The search space explored would require over 11,000 combinations exhaustively, whereas we use 10 TPE trials are sufficient to identify well-performing configurations. The parameters searched, their types, and ranges are summarised in Table 1 in the appendix, as well as further model training details.

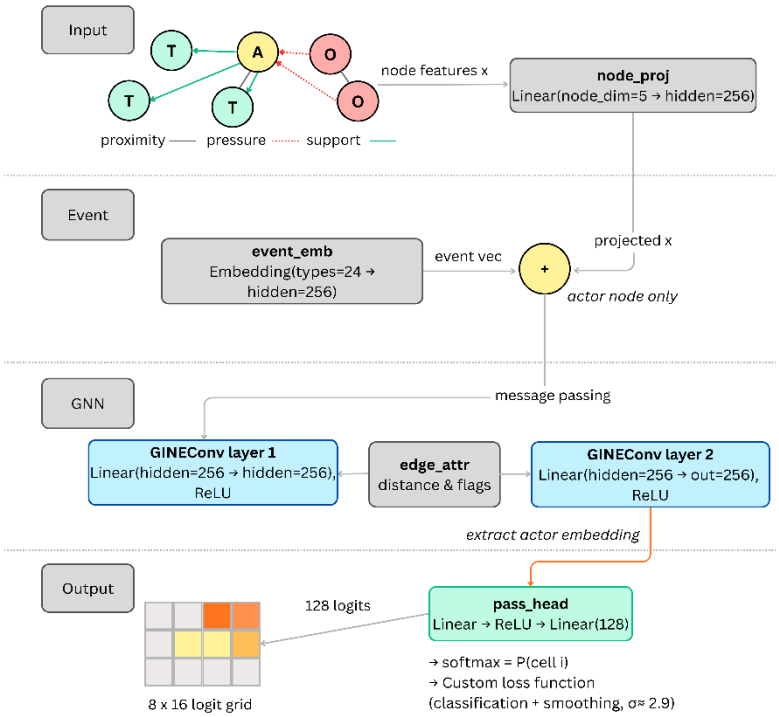

Figure 4 — PassPredictionGNN Model Architecture

3.3 Prediction Visualization

After training, pass predictions are visualised as spatial heatmaps overlaid on a StatsBomb-dimensioned pitch (120 × 80 m). For each passing event, the model outputs 128 logits over the pitch grid (this discreteness is a configuration that follows from the hyperparameter tuning). The logits are converted to probabilities via softmax, reshaped into a 2D grid, and smoothed again with a Gaussian filter. The visualization smoothing just interpolates between those discrete values to produce a visually continuous surface. The predicted destination is taken as the argmax of the smoothed heatmap, then denormalized to pitch coordinates. The inference pipeline is denoted in Figure 5.

3.4 Evaluation Metrics

Model performance is assessed on a held-out test set of 43,224 passing events, using 5 different measures denoted in Table 1. All metrics are compared against two baselines: a uniform random predictor, which assigns equal probability to all cells (expected rank: 64/128), and a centre-of-pitch

predictor, which always predicts the pitch centre (ADE: 33.96 m) as the pass destination regardless of game state. These baselines establish the floor above which the model must perform to demonstrate meaningful learning.

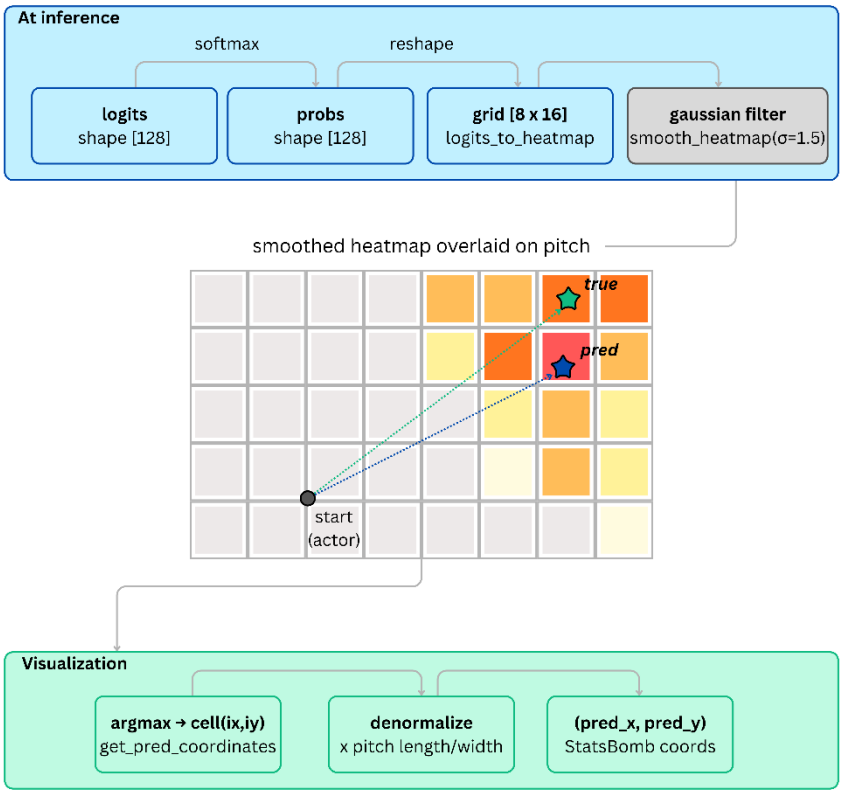

Figure 5 — Inference Pipeline

Measure	**Metric**	**Description**
Spatial Accuracy	*Average Displacement Error (ADE)*	Measures the Euclidean distance in metres between the predicted destination and the true destination, reported using both the argmax cell peak and the probability-weighted centroid of the full distribution. The centroid variant is generally lower, as it accounts for distributional spread rather than committing to a single cell.
Distribution Quality	*Top-k Accuracy*	Measures the fraction of predictions in which the true destination cell appears among the model's k highest-probability cells, reported for $k \in \{1,3,5\}$.
	Mean/Median Rank	Sorts cells by predicted probability and finds where the true destination lands in ranking. Reported as both mean and median, providing a distribution-level measure of how

		well the model concentrates probability mass around the correct region.
Baselines	*Uniform Random Predictor*	Picks any cell with equal probability.
	Centre-of-pitch Predictor	Always guesses middle of field.

Table 1 — Evaluation Metrics

4) Results

4.1 Quantitative Evaluation

Metric	Model	Centre-of-pitch	Random
ADE: argmax (m)	20.23	33.96	—
ADE: centroid (m)	18.29	33.96	—
Top-1 Accuracy	7.3%	—	0.8%
Top-3 Accuracy	20.9%	—	2.3%
Top-5 Accuracy	32.3%	—	3.9%
Mean Rank / 128	14.9	—	64.0
Median Rank / 128	9.0	—	64.0

Table 2 — Test set evaluation results

The model achieves a median rank of 9.0 out of 128 cells, placing the true pass destination in the top 7% of all predicted cells on average. This represents a 7× improvement over the random baseline median rank of 64. The mean rank of 14.9 is higher than the median, indicating a right-skewed distribution of errors, meaning that the model predicts the majority of passes with high confidence in the correct region, with a smaller number of larger errors pulling the mean upward. Top-5 accuracy of 32.3% demonstrates that more than one in three passes have their true destination within the model's five most probable cells, compared to 3.9% under random chance.

Against the centre-of-pitch baseline ADE of 33.96 m, the centroid ADE represents a 46% reduction in average displacement error, demonstrating that the model has learned meaningful spatial structure from the player graph. The centroid ADE of 18.00 m is 1.94 m lower than the argmax ADE of 20.79 m, confirming that the model produces well-spread probability distributions rather than collapsing onto a single cell. The gap between the two ADE variants reflects a model that reasons about spatial regions of likely destinations rather than making hard point predictions. The centroid ADE is a perhaps more insightful metric than the argmax variant, because it incorporates the uncertainty of the model. For example, if the model puts 20% probability on a cell at x=60 and 20% on a cell at x=70, the argmax picks one arbitrarily, landing metres away from the other. The centroid gives x=65, so closer to both. Across many predictions this consistently reduces ADE because the measure is no longer throwing away all the probability mass that didn't win the argmax.

4.2 Qualitative Analysis

Below are examples of PassPredictionGNN model predictions on the validation set, each displaying the player configuration graph (left) and predicted pass heatmap (right).

In cases where the prediction is relatively accurate (like in Figures 6 and 7) the heatmap shows a concentrated warm region aligned with the true destination. Even when the argmax cell does not precisely match the true destination (like in Figure 8), the probability mass is concentrated in the correct region of the pitch. This is reflected in the gap between top-1 accuracy (7.3%) and top-5 accuracy (32.3%): the model is directionally correct more often than it is exactly correct. This behaviour is expected given the inherent ambiguity of passing decisions, where multiple destinations may be equally viable from a given game state. In this same vein, there are predictions that are far from the true destination of the pass, such as Figure 9. As further discussed in the following section, pass destination prediction is a fundamentally difficult problem, so this is by no means a perfect model and as such, there will be some predictions that are considerably far from the ground truth.

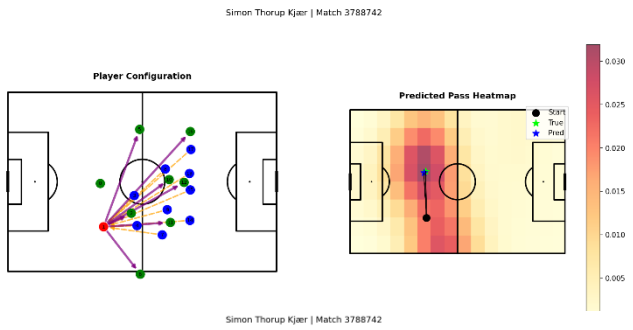

Figure 6

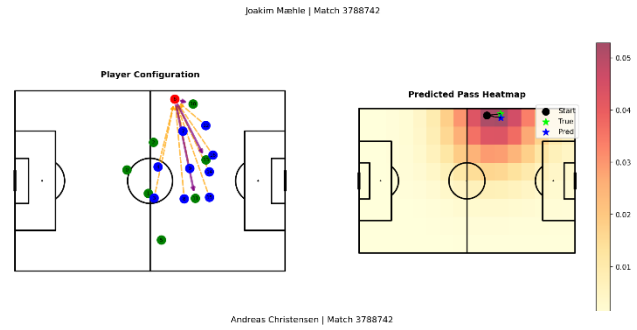

Figure 7

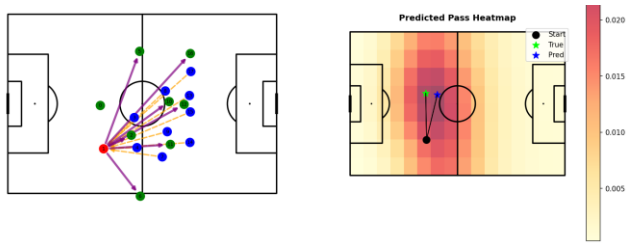

Figure 8

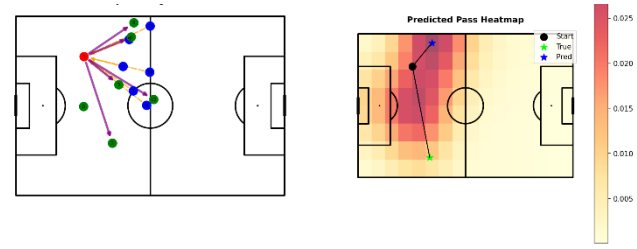

Figure 9

4.3 Discussion

The evaluation results demonstrate that modelling passing behaviour as a graph classification problem captures meaningful structure in player spatial relationships. The rank-based metrics are the strongest signal: a median rank of 9/128 indicates the model consistently concentrates probability in the correct region of the pitch.

Pass destination prediction is fundamentally a one-to-many problem. From any given game state, a soccer player typically has several tactically reasonable options, and the correct choice depends on many factors that are not fully observable from spatial snapshots alone. This model is not attempting to recover a deterministic function from game state to destination, but to learn the distribution of likely destinations given the observable spatial configuration.

This distinction matters for how results should be interpreted. A top-5 accuracy of 32.3% does not mean the model is wrong 64.5% of the time — it means the true destination fell outside the five

highest-probability cells in those cases, which may simply reflect that the player made a lower-probability but tactically sound choice. Rather than collapsing onto a single predicted destination but spreading probability mass across the plausible region of the pitch. This is also why rank-based metrics are an appropriate primary evaluation measure for this task. Rather than asking whether the model picked the exact cell, rank asks how highly the model rated the true outcome relative to all others.

The primary limitation of the current model is that it conditions on the freeze frame at the moment of the pass, without considering the sequence of events leading up to it. Incorporating temporal context, such as the preceding events, would likely improve predictions in transitional play where passing patterns depend on recent ball movement. Additionally, the pitch grid discretization introduces irreducible quantization error in the ADE metric.

5) Next Steps

The current model operates at the event level, i.e., each passing situation produces a prediction for a single play. However, the actor node's final embedding (the representation that feeds into the prediction head) encodes something richer than a single pass destination: it captures how that player tends to behave, conditioned on their spatial context. By aggregating these embeddings across all passing events for a given player, it becomes possible to construct a player-level representation that summarises their passing style. A player who consistently receives high-probability mass in forward channels looks different in embedding space from one whose distributions concentrate laterally or backward. Players with similar tactical tendencies should cluster together; stylistically distinct players should be separable.

This aggregation step is the natural next question this work enables: not "where will this pass go?" but "how does this player pass?", and by extension, "which players pass similarly?" This opens the door to player similarity search, role clustering, and tactical profiling across competitions and seasons, which was the original motivation stated at the outset of the project.

Beyond player embeddings, several extensions would strengthen the model itself:

- Temporal context: incorporating the sequence of events preceding the pass would help in transitional situations where passing patterns depend on recent ball movement
- Attention mechanisms: replacing GINEConv with a graph attention network (GAT) would allow the model to learn which neighbouring players are most relevant for each decision, rather than treating all edges equally
- Extended event types: applying the same framework to shots, carries, and defensive actions would enable a unified event-level representation across the full match

6) Appendix

6.1 Further Details about Graph Construction

Each passing event is transformed into a PyTorch torch_geometric.data.Data object, where the nodes represent individual players and edges encode the spatial relationships between them.

The end location from the event table provides the pass destination label used during training.

```
Data(
        x,                  # node features
        edge_index,         # graph connectivity
        edge_attr,          # edge features
        actor_idx,          # index of the event-making player
        actor_player_id,    # StatsBomb player ID
        event_type_idx,     # encoded event type
        y,                  # pass end location
        metadata
    )
```

Each node corresponds to one player visible in the freeze frame and encodes both spatial and semantic information, where:

```
node_features = [x, y, is_teammate, is_actor, is_keeper]
```

- (x, y) are the player's on-pitch coordinates
- Binary flags identify team membership, goalkeepers, and the actor

In our model, we assign each edge a 4D feature vector:

```
edge_attr = [distance, is_proximity, is_pressure, is_support]
```

6.2 Graph Isomorphism Network with Edge features

Edges encode pairwise relationships between players. In this representation, we assume that pairwise relationships are an adequate approximation of the local tactical context, even though football behavior can involve higher-order or team-level structure. Rather than having an edge just represent a connection between two nodes, we can enrich the edges with additional information, i.e., $i - j$ with attributes $e_{ij} \in \mathbb{R}^k$.

In a basic GNN, updates look like:

$$h_i' = MLP\left((1+\epsilon)\cdot h_i + \sum_{j\in\mathcal{N}(i)} h_j\right)$$

Node i looks at all its neighbours and aggregates their embeddings, then applies a learned Neural Network function. All nonlinearity occurs after aggregation. However, since we want to incorporate tactical information between players, we can use a more sophisticated PyTorch model, Graph Isomorphism Network with Edge features (GINEConv). GINEConv explicitly injects edge features into the message passing process before activation.

Let

$$\begin{cases} h_i\text{: node feature} \\ e_{i,j}\text{: edge feature for edge } (i,j) \\ \phi\text{: MLP for final update} \\ ReLU\text{: activation function} \end{cases}$$

The update rule becomes:

$$h_i' = \phi\left((1+\epsilon)\cdot h_i + \sum_{j\in\mathcal{N}(i)} ReLU\left(h_j + e_{j,i}\right)\right)$$

Multi-dimensional edge attributes allow the model to represent different types of player interactions within a single graph, and GINEConv incorporates these attributes directly into message passing, enabling the network to learn how different spatial and tactical relationships influence passing decisions.

6.3 Hyperparameter Tuning with Optuna

Parameter	Type	Range/Choices
hidden_dim	Categorical	64,128,256
out_dim	Categorical	64,128,256
batch_size	Categorical	32,64,128
grid_x	Categorical	12,16,24
grid_y	Categorical	8,12,16
lr	Log-uniform float	[0.0001,0.01]
sigma	Uniform float	[0.5,3.0]
epochs	Integer	[5,20]

Table 1 — Hyperparameter search space

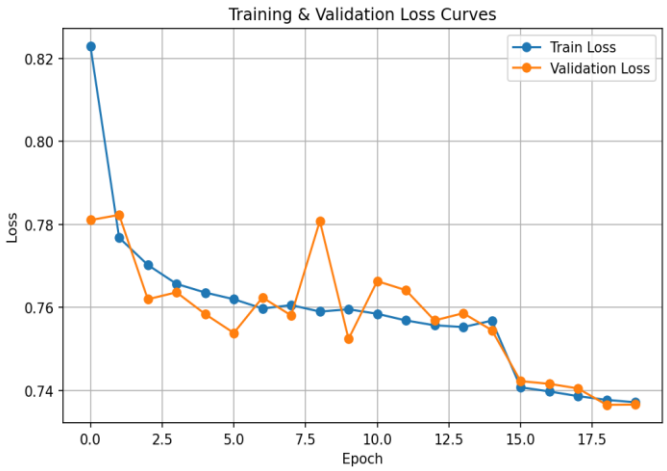

Figure 1 —Model training results

The best configuration found was hidden_dim=256, out_dim=256, grid_x=16, grid_y=8, lr=0.0079, sigma=2.91, batch_size=64, epochs=20.The model was trained using the Adam optimiser with a ReduceLROnPlateau scheduler, which halves the learning rate when validation loss plateaus for lr_patience consecutive epochs. Early stopping was applied with a patience of 10 epochs. Parameters controlling training stability (lr_patience, es_patience) were held fixed at 4 and 10 respectively. The custom loss function computes the loss between the model's output logits and a target cell index derived the true pass destination coordinates as follows:

1. Build a one-hot target
    - Vector of the same shape as the logits ([B, 128]), with a 1.0 at the true cell index for each sample in the batch.
2. Apply Gaussian smoothing
    - The one-hot grid is reshaped to 2D ([grid_y, grid_x]) and a Gaussian filter is applied. Cells adjacent to the true destination get some probability, cells further away get less, and the falloff follows a Gaussian curve.
3. Renormalize
    - After smoothing, the values across all cells no longer sum to 1.0 exactly, so dividing by the sum restores a valid probability distribution.
4. Compute the KL divergence
    - KL divergence measures the difference between the model's predicted distribution and the soft target distribution (taking log soft max of the logits).

The motivation behind this loss function is that the model is trained to understand that pass destinations exist in continuous space, not as 128 completely independent discrete classes. The sigma parameter controls the Gaussian smoothing; it encourages the model to spread probability mass across spatially adjacent cells rather than treating prediction as a hard single-cell classification problem A higher sigma means a wider, flatter distribution; a lower sigma means a tighter peak around the true cell.